

Hello There !!!

Team Members:

Isaiah Buluran, Anmol Nayak, Riley
Blacklock



Clothing Store Management Database System

- Retail clothing store database
- Designed for inventory, sales, RFID tracking, and employee management
- Built using normalized relational database design
- Cloud-deployed database with live queries and migrations



Project Topic:

A centralized clothing store management system for retail operations.

Real-World Use Case

The database helps retail stores manage:

- RFID tracking
- Customer purchases
- Sales transactions
- Employee activity
- Restocking and suppliers
- Inventory reports



- Store Owners



Managers:
(yes i am a
manager)

Employees



MAIN USERS:



System Features:



- Store clothing item information
- Track inventory levels
- Process customer purchases
- Track RFID-tagged products
- Monitor employee sales activity
- Handle supplier restocking
- Generate sales reports
- Support cloud database deployment



ER Design Decisions Continued:

Order_Items: Allows one order to contain multiple products.

Inventory

Tracks quantity available and restocking needs.

Suppliers

Stores supplier information for inventory restocking.

Sales

Stores payment and transaction information.



Key Relationships:

Clothing_Items → Inventory: Each clothing item has inventory tracking.

Suppliers → Clothing_Items: Many suppliers can provide many products.

Customer → Orders

One customer can place many orders.

Employee → Orders

One employee processes many customer orders.

Orders → Order_Items

One order can contain many products.

Primary Keys and Foreign Keys



Primary Keys

Each table uses a unique identifier.

Examples:

employee_id
customer_id
order_id
item_id

Foreign Keys

Used to connect related tables.

Examples:

orders.customer_id → customers.customer_id

orders.employee_id → employees.employee_id

order_items.order_id → orders.order_id

READ Query #1

Retrieve all customer orders.

```
SELECT customers.first_name,  
       orders.order_id,  
       orders.order_date  
FROM customers  
JOIN orders  
ON customers.customer_id =  
   orders.customer_id;
```



first_name	order_id	order_date
John	101	2026-05-01
Sarah	102	2026-05-03

READ Query #2

Retrieve clothing inventory levels.

```
SELECT  
clothing_items.item_name,  
  
inventory.quantity_available  
FROM clothing_items  
JOIN inventory  
ON clothing_items.item_id =  
inventory.item_id;
```



item_name	quantity_available
Hoodie	15
Jeans	8

UPDATE Query #1

Update inventory quantity.

```
UPDATE inventory  
SET quantity_available = 20  
WHERE item_id = 1;
```



Inventory
successfully
updated!!!

UPDATE Query #2

Update inventory quantity.

```
UPDATE customers  
SET loyalty_id = 2  
WHERE customer_id = 5;
```



Customer
loyalty level
changed
successfully.

DELETE Query #1

Delete a cancelled order item.

```
DELETE FROM order_items  
WHERE order_item_id = 15;
```



Order item
removed
successfully.

DELETE Query #2

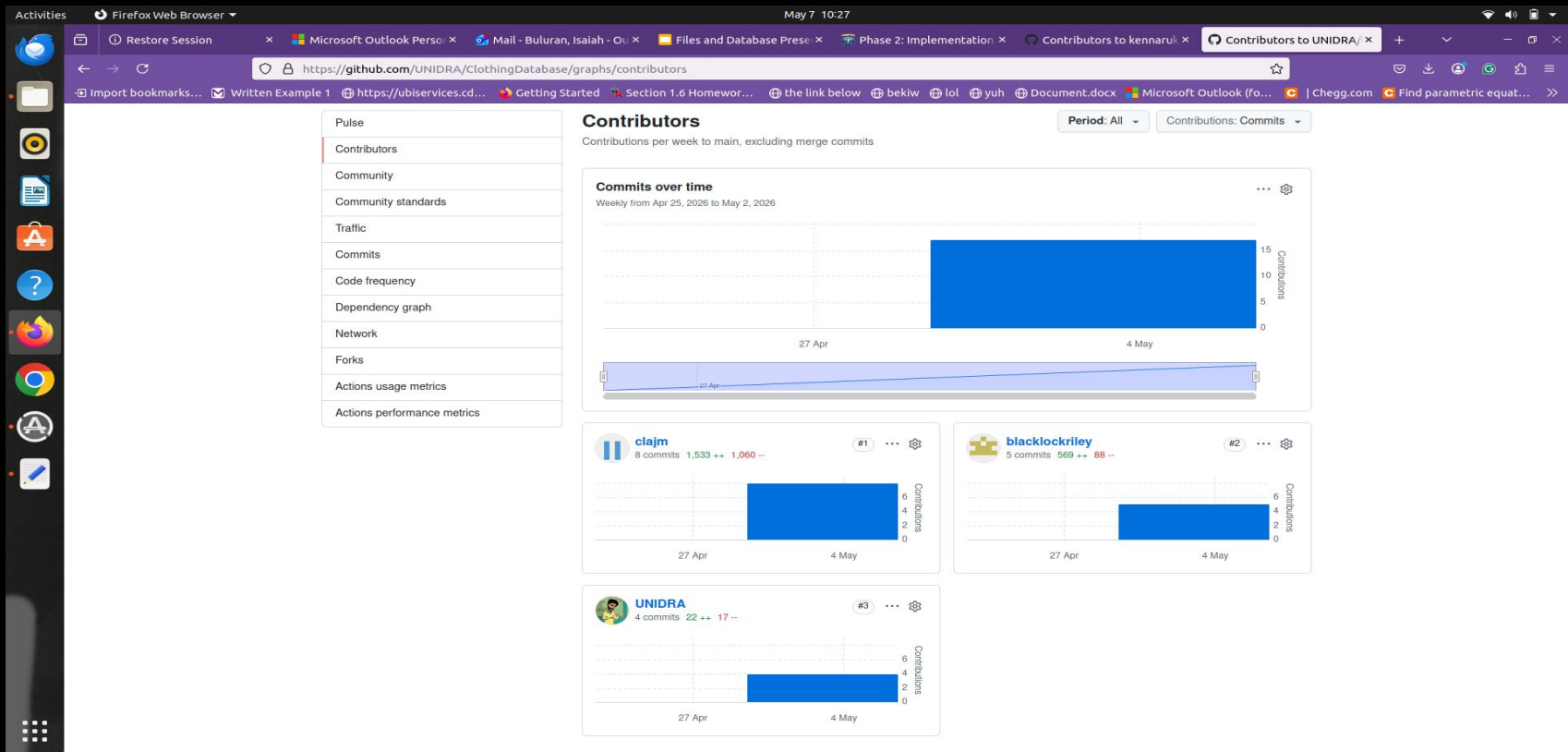
Delete inactive supplier.

```
DELETE FROM suppliers  
WHERE supplier_id = 4;
```



Supplier
removed
successfully.

GitHub Contributors:



Cloud Demos:

THANK YOU!

<https://github.com/UNIDRA/ClothingDatabase>

Any Questions?